

AFRILANG Stdlib

std/c/crypthmac

Bibliothèque AFRILANG `std/c/crypthmac` (niveau complex, catégorie complex). Elle expose 8 fonction(s) :
hmacLen, hmacUp

```
`import "std/c/crypthmac" . `use crypthmac`
```

- `hmacLen(s text) ? number`
- `hmacUpper(s text) ? text`
- `hmacLower(s text) ? text`
- `hmacTrim(s text) ? text`
- `hmacSplit(s text, delim text) ? list text`
- `hmacJoin(parts list text, delim text) ? text`
- `hmacReplace(s text, from text, to text) ? text`
- `hash32(s text) ? number`

Category: complex | Tier: complex | Functions: 8

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/crypthmac` (niveau complex, catégorie complex). Elle expose 8 fonction(s) : hmacLen, hmacUp

```
`import "std/c/crypthmac" . `use crypthmac`  
- `hmacLen(s text) ? number`  
- `hmacUpper(s text) ? text`  
- `hmacLower(s text) ? text`  
- `hmacTrim(s text) ? text`  
- `hmacSplit(s text, delim text) ? list text`  
- `hmacJoin(parts list text, delim text) ? text`  
- `hmacReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/crypthmac"  
use crypthmac
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? hmacLen(s text) ? number  
? hmacUpper(s text) ? text  
? hmacLower(s text) ? text  
? hmacTrim(s text) ? text  
? hmacSplit(s text, delim text) ? list  
? hmacJoin(parts list text, delim text) ? text  
? hmacReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Exemples d'utilisation

```
import "std/c/crypthmac"  
use crypthmac  
  
create result = hmacLen(/* args */)   
say result  
  
create result = hmacUpper(/* args */)   
say result  
  
create result = hmacLower(/* args */)   
say result  
  
create result = hmacTrim(/* args */)   
say result  
  
create result = hmacSplit(/* args */)   
say result  
  
create result = hmacJoin(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/crypthmac"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module crypt hmac
function hmacLen(s text) returns number
end
function hmacUpper(s text) returns text
end
function hmacLower(s text) returns text
end
function hmacTrim(s text) returns text
end
function hmacSplit(s text, delim text) returns list text
end
function hmacJoin(parts list text, delim text) returns text
end
function hmacReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/crypthmac` (tier complex, category complex). Exports 8 function(s): hmacLen, hmacUpper, hmacLower

```
`import "std/c/crypthmac" . `use crypthmac`  
- `hmacLen(s text) ? number`  
- `hmacUpper(s text) ? text`  
- `hmacLower(s text) ? text`  
- `hmacTrim(s text) ? text`  
- `hmacSplit(s text, delim text) ? list text`  
- `hmacJoin(parts list text, delim text) ? text`  
- `hmacReplace(s text, from text, to text) ? text`  
- `hash32(s text) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/crypthmac"  
use crypthmac
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? hmacLen(s text) ? number  
? hmacUpper(s text) ? text  
? hmacLower(s text) ? text  
? hmacTrim(s text) ? text  
? hmacSplit(s text, delim text) ? list  
? hmacJoin(parts list text, delim text) ? text  
? hmacReplace(s text, from text, to text) ? text  
? hash32(s text) ? number
```

4. Usage examples

```
import "std/c/crypthmac"  
use crypthmac  
  
create result = hmacLen(/* args */)   
say result  
  
create result = hmacUpper(/* args */)   
say result  
  
create result = hmacLower(/* args */)   
say result  
  
create result = hmacTrim(/* args */)   
say result  
  
create result = hmacSplit(/* args */)   
say result  
  
create result = hmacJoin(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/crypthmac"` at the top of your file.  
? Run `afrilang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module crypt hmac
function hmacLen(s text) returns number
end
function hmacUpper(s text) returns text
end
function hmacLower(s text) returns text
end
function hmacTrim(s text) returns text
end
function hmacSplit(s text, delim text) returns list text
end
function hmacJoin(parts list text, delim text) returns text
end
function hmacReplace(s text, from text, to text) returns text
end
function hash32(s text) returns number
end
end
```